

CyberAudit

Audit Pack & Request API

4 packs commerciaux · CRUD audit requests · RBAC filtering · Notifications

Project / Projet	CyberAudit Platform
Module	apps/audits/ models.py + views.py + serializers.py + urls.py
Models	AuditPack (catalogue) + AuditRequest (demande client)
Packs	Audit, security, protection, premium (ordered by price)
Reference format	DOSSIER-YYYY-NNNN: auto-generated, atomic, collision-free
RBAC	Client : sees own requests only, Admin : sees all requests
Status lifecycle	pending → in_progress → completed, archived (soft-delete)
Notifications	create_and_send() called on request creation + status change
Pack endpoints	GET /api/packs/, GET /api/packs/{id}/, AllowAny (public)
Audit endpoints	GET POST /api/audits/ · GET PATCH DELETE /api/audits/{uuid}/
Tests	20 tests across 7 classes (models + all endpoints)
Date	June 2026 / Juin 2026

1. Data Models

1. Modèles de données

The audits app defines two models: AuditPack (the fixed commercial catalogue, 4 packs seeded at deployment) and AuditRequest (a client's request, tracked through a status lifecycle). Both use ForeignKey with PROTECT to prevent orphaned records.

L'app audits définit deux modèles : AuditPack (le catalogue commercial fixe, 4 packs implantés au déploiement) et AuditRequest (une demande client, suivie selon un cycle de statuts). Les deux utilisent ForeignKey avec PROTECT pour éviter les enregistrements orphelins.

AuditPack : commercial catalogue / catalogue commercial

```
class AuditPack(models.Model):
    class Code(models.TextChoices):
        AUDIT = 'audit', 'Audit'
        SECURITY = 'security', 'Security'
        PROTECTION = 'protection', 'Protection'
        PREMIUM = 'premium', 'Premium'

    code = models.CharField(max_length=20, choices=Code.choices,
unique=True)
    name = models.CharField(max_length=50)
    description = models.TextField(blank=True)
    included_services = models.TextField(blank=True)
    for_whom = models.TextField(blank=True)
    perimeter = models.TextField(blank=True)
    duration_days = models.PositiveSmallIntegerField()
    price = models.DecimalField(max_digits=8, decimal_places=2)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'audit_packs'
        ordering = ['price'] # cheapest first in all list responses
```

AuditPack fields / Champs AuditPack

Field	Type	Purpose EN / FR
code	CharField (unique)	(EN) Machine identifier: audit / security / protection / premium. (FR) Identifiant machine : audit / security / protection / premium.
name	CharField	(EN) Display name shown in UI and emails. E.g. 'Audit Pack'. (FR) Nom affiché dans l'UI et les emails. Ex. 'Audit Pack'.
description	TextField	(EN) Marketing description of the pack. (FR) Description marketing du pack.
included_services	TextField	(EN) List of services included in this pack (free text). (FR) Liste des services inclus dans ce pack (texte libre).

for_whom	TextField	(EN) Target audience description. (FR) Description de l'audience cible.
perimeter	TextField	(EN) Technical perimeter covered by the audit. (FR) Périmètre technique couvert par l'audit.
duration_days	PositiveSmallInt	(EN) Estimated delivery time in business days. (FR) Délai de livraison estimé en jours ouvrés.
price	DecimalField(8,2)	(EN) Price in EUR. Displayed ascending (ordering=['price']). (FR) Prix en EUR. Affiche croissant (ordering=['price']).

AuditRequest : client request / demande client

```
class AuditRequest(models.Model):
    class Status(models.TextChoices):
        PENDING      = 'pending',      'Pending'
        IN_PROGRESS   = 'in_progress',  'In Progress'
        COMPLETED    = 'completed',    'Completed'
        ARCHIVED      = 'archived',     'Archived'

    id              = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
    reference       = models.CharField(max_length=20, unique=True, db_index=True,
blank=True)
    client          = models.ForeignKey(AUTH_USER_MODEL, on_delete=PROTECT,
related_name='audit_requests')
    pack            = models.ForeignKey(AuditPack, on_delete=PROTECT,
related_name='requests')
    status          = models.CharField(choices=Status.choices, default=Status.PENDING,
db_index=True)
    scope_notes     = models.TextField(blank=True) # client describes their context
    assigned_to     = models.ForeignKey(AUTH_USER_MODEL, on_delete=SET_NULL,
null=True)
    internal_notes  = models.TextField(blank=True) # admin-only notes
    submitted_at    = models.DateTimeField(auto_now_add=True)
    updated_at      = models.DateTimeField(auto_now=True)
    completed_at    = models.DateTimeField(null=True, blank=True)

    class Meta:
        db_table = 'audit_requests'
        ordering = ['-submitted_at'] # newest first
        indexes = [models.Index(fields=['client', '-submitted_at'])]
```

AuditRequest key fields / Champs clés AuditRequest

id (UUID PK)	(EN) UUIDField, editable=False prevents IDOR attacks on request lookup. (FR) UUIDField, editable=False prévient les attaques IDOR sur la recherche de demande.
reference	(EN) Auto-generated DOSSIER-YYYY-NNNN. Written atomically in save() using select_for_update() to prevent collisions under concurrency. (FR) Génère automatiquement DOSSIER-YYYY-NNNN. Ecrit de manière atomique dans save() via select_for_update() pour éviter les collisions en concurrence.

client FK (PROTECT)	(EN) on_delete=PROTECT prevents deleting a user who has requests. Preserves audit history. (FR) on_delete=PROTECT empêche la suppression d'un utilisateur ayant des demandes. Preserve l'historique d'audit.
pack FK (PROTECT)	(EN) on_delete=PROTECT prevents deleting a pack referenced by requests. (FR) on_delete=PROTECT empêche la suppression d'un pack reference par des demandes.
assigned_to FK (SET_NULL)	(EN) on_delete=SET_NULL if the assigned admin is deleted, field becomes null. (FR) on_delete=SET_NULL si l'admin assigne est supprime, le champ devient null.
internal_notes	(EN) Admin-only field never exposed in AuditRequestCreateSerializer or client-facing views. (FR) Champ admin uniquement jamais expose dans AuditRequestCreateSerializer ou les vues client.
status (db_index)	(EN) Indexed for fast filtering in admin dashboard. (FR) Indexe pour un filtrage rapide dans le tableau de bord admin.

2. Reference Auto-Generation: DOSSIER-YYYY-NNNN

2. Génération automatique de référence : DOSSIER-YYYY-NNNN

Every AuditRequest gets a human-readable unique reference automatically on first save. The algorithm uses a database transaction with SELECT FOR UPDATE to guarantee no two concurrent requests can receive the same number, even under load.

Chaque AuditRequest recoit une référence unique lisible automatiquement au premier save.

L'algorithme utilise une transaction DB avec SELECT FOR UPDATE pour garantir qu'aucune deux requêtes concurrentes ne peuvent recevoir le même numéro, même sous charge.

```
def save(self, *args, **kwargs):
    if not self.reference:
        self.reference = self._generate_reference()
    super().save(*args, **kwargs)

    @classmethod
    def _generate_reference(cls):
        year = datetime.now().year
        prefix = f'DOSSIER-{year}-'
        with transaction.atomic():
            last = (
                cls.objects.select_for_update()           # LOCK the matching rows
                .filter(reference__startswith=prefix)
                .order_by('-reference')                    # lexicographic → last number
                .first()
            )
            next_num = (int(last.reference.split('-')[-1]) + 1) if last else 1
        return f'{prefix}{next_num:04d}' # e.g. DOSSIER-2026-0001
```

blank=True on field	(EN) Allows save() to be called without a reference; the method fills it before INSERT. (FR) Permet d'appeler save() sans référence ; la méthode la renseigne avant l'INSERT.
select_for_update()	(EN) Locks the filtered rows until the transaction commits. No other transaction can insert/modify while the counter is being read. Verrouille les lignes filtrées jusqu'à la validation de la transaction. (FR) Aucune autre transaction ne peut insérer/modifier pendant la lecture du compteur.
Atomicity	(EN) transaction.atomic() wraps the read+compute step. Rollback on error prevents gaps. (FR) transaction.atomic() encapsule l'étape lecture+calcul. Rollback en cas d'erreur prévient les trous.
Format :04d	(EN) Left-pads with zeros to 4 digits: 0001, 0002 ... 9999. Easily sortable. (FR) Rembourrures zéros à 4 chiffres : 0001, 0002 ... 9999. Facilement triable.
Year in prefix	(EN) New year resets numbering automatically (2026-0001, 2027-0001...). (FR) La nouvelle année réinitialise la numérotation automatiquement (2026-0001, 2027-0001...).

3. Serializers : 4 classes

3. Sérialiseurs : 4 classes

The app uses four serializers: AuditPackSerializer (read-only public), AuditRequestSerializer (full read, owner or admin), AuditRequestCreateSerializer (client POST, minimal writable fields), and AuditRequestAdminUpdateSerializer (admin PATCH, status + notes).

L'app utilise quatre sérialiseurs : AuditPackSerializer (lecture seule public), AuditRequestSerializer (lecture complete, propriétaire ou admin), AuditRequestCreateSerializer (POST client, champs modifiables minimaux), et AuditRequestAdminUpdateSerializer (PATCH admin, statut + notes).

Serializer	Used by	Fields / purpose EN+FR
AuditPackSerializer	PackListView PackDetailView	(EN) Read-only. 9 fields: id, code, name, description, included_services, for_whom, perimeter, duration_days, price. Exposed publicly (AllowAny). (FR) Lecture seule. 9 champs. Expose publiquement (AllowAny).
AuditRequestSerializer	GET /audits/ GET /audits/{id}/	(EN) Full read. Nested AuditPackSerializer + client_info SerializerMethodField. All fields read_only. Shows internal_notes. Lecture complete. (FR) Pack imbriqué + client_info SerializerMethodField. Tous champs read_only. Affiche internal_notes.
AuditRequestCreateSerializer	POST /audits/	(EN) Client creates request. Writable: pack, scope_notes. Read-only: id, reference, status, submitted_at. client is injected by view. (FR) Client crée la demande. Modifiables : pack, scope_notes. En lecture : id, reference, status, submitted_at. client injecte par la vue.
AuditRequestAdminUpdateSerializer	PATCH /audits/{id}/	(EN) Admin-only update. Writable: status, internal_notes, assigned_to, completed_at. Client fields not exposed. (FR) Mise à jour admin uniquement. Modifiables : status, internal_notes, assigned_to, completed_at. Champs client non exposes.

client_info SerializerMethodField : custom nested read

```
class AuditRequestSerializer(serializers.ModelSerializer):
    pack = AuditPackSerializer(read_only=True) # nested full pack
    client_info = serializers.SerializerMethodField() # custom dict

    def get_client_info(self, obj):
        return {
            'email': obj.client.email,
            'first_name': obj.client.first_name,
            'last_name': obj.client.last_name,
            'company_name': obj.client.company_name,
        }
# → Admin sees who submitted the request without exposing the full user UUID
# → select_related('client') in get_queryset() avoids N+1 queries
```

4. URL Routing

4. Routage URL

```
# audits/urls.py – prefixed in config/urls.py with /api/
from .views import PackListView, PackDetailView,
                  AuditRequestListCreateView, AuditRequestDetailView

urlpatterns = [
    path('packs/', PackListView.as_view(), name='pack-
list'),
    path('packs/<int:pk>/', PackDetailView.as_view(), name='pack-
detail'),
    path('audits/', AuditRequestListCreateView.as_view(), name='audit-
list-create'),
    path('audits/<uuid:pk>/', AuditRequestDetailView.as_view(), name='audit-
detail'),
]
# Note: <uuid:pk> – Django validates UUID format automatically (400 on bad UUID)
```

URL	Method	Auth	Status codes	Description EN / FR
GET /api/packs/	GET	None	200	(EN) List all 4 packs ordered by price. Paginated. (FR) Liste des 4 packs ordonnées par prix. Page.
GET /api/packs/{id}/	GET	None	200, 404	(EN) Detail of a single pack by integer PK. (FR) Détail d'un pack par cle primaire entière.
GET /api/audits/	GET	Bearer	200, 401	(EN) Client: own requests. Admin: all requests. Paginated. (FR) Client : ses propres demandes. Admin : toutes. Page.
POST /api/audits/	POST	Bearer	201, 401, 403	(EN) Client creates a new audit request. Admin → 403. (FR) Le client crée une nouvelle demande. Admin → 403.
GET /api/audits/{uuid}/	GET	Bearer	200, 401, 404	(EN) Owner or admin reads full detail. Other client → 404. (FR) Propriétaire ou admin lit le détail. Autre client → 404.
PATCH /api/audits/{uuid}/	PATCH	Bearer (admin)	200, 401, 403	(EN) Admin updates status / notes / assigned_to. Client → 403. (FR) Admin met à jour statut / notes / assignation. Client → 403.
DELETE /api/audits/{uuid}/	DELETE	Bearer (admin)	204, 401, 403	(EN) Admin soft-deletes → sets status='archived'. Row kept in DB. (FR) Admin soft-delete → passe status='archived'. Ligne gardée en DB.

5. Views & RBAC Logic

5. Vues et logique RBAC

The RBAC logic is applied in `get_queryset()` (data scoping) and in update/destroy guards (permission checks). Clients are completely isolated from each other's requests through queryset filtering, they receive 404 (not 403) when accessing another client's request, which prevents enumeration.

La logique RBAC est appliquée dans `get_queryset()` (périmètre des données) et dans les gardes `update/destroy` (vérification des permissions). Les clients sont complètement isolés des demandes des autres via le filtrage du queryset, ils reçoivent 404 (pas 403) lors d'un accès à la demande d'un autre client, ce qui prévient l'énumération.

AuditRequestListView : GET (RBAC) + POST (client only)

```
class AuditRequestListView(generics.ListCreateAPIView):
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        qs = AuditRequest.objects.select_related('pack', 'client', 'assigned_to')
        if self.request.user.role == 'admin':
            return qs # admin sees ALL requests
        return qs.filter(client=self.request.user) # client sees only own

    def get_serializer_class(self):
        if self.request.method == 'POST':
            return AuditRequestCreateSerializer # limited fields for creation
        return AuditRequestSerializer # full read

    def perform_create(self, serializer):
        if self.request.user.role == 'admin':
            raise PermissionDenied('Only clients can submit a request.') # 403
        audit = serializer.save(client=self.request.user) # inject client
        create_and_send( # automatic acknowledgement notification + email
            user=audit.client,
            request=audit,
            type_=Notification.Type.REQUEST_RECEIVED,
            subject=f'Audit request received - {audit.reference}',
            message=f'Hello {audit.client.first_name}, ...',
        )
```

AuditRequestDetailView : GET / PATCH (admin) / DELETE → archive (admin)

```
class AuditRequestDetailView(generics.RetrieveUpdateDestroyAPIView):
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        qs = AuditRequest.objects.select_related('pack', 'client', 'assigned_to')
        if self.request.user.role == 'admin':
            return qs
        return qs.filter(client=self.request.user) # 404 if not owner (not 403!)

    def update(self, request, *args, **kwargs):
        if request.user.role != 'admin':
            raise PermissionDenied('Only an administrator can modify a request.')
# 403
        return super().update(request, *args, **kwargs)

    def perform_update(self, serializer):
        old_status = serializer.instance.status # capture before save
        audit = serializer.save()
        if old_status != audit.status: # only notify on real change
            create_and_send( # status-change email to client
                user=audit.client, request=audit,
                type_=Notification.Type.STATUS_CHANGED, ...
            )

    def perform_destroy(self, instance):
        if self.request.user.role != 'admin':
            raise PermissionDenied('Only an administrator can archive a request.')
# 403
        # Soft-delete - row is kept, only status changes
        instance.status = AuditRequest.Status.ARCHIVED
        instance.save(update_fields=['status', 'updated_at'])
```

RBAC matrix / Matrice RBAC

Action	Client (role='client')	Admin (role='admin')
GET /api/packs/	(EN) 200: public, no auth needed / (FR) 200: public, sans auth	(EN)200: public, no auth needed / (FR) 200: public, sans auth
POST /api/audits/	(EN) 201 : creates request, client injected / (FR) 201 : crée la demande, client injecte	(EN) 403 : PermissionDenied / (FR) 403 : PermissionRefusée
GET /api/audits/ (list)	(EN) 200 : own requests only / (FR) 200 : ses demandes uniquement	(EN) 200 : all requests / (FR) 200 : toutes les demandes
GET /api/audits/{id}/ (own)	(EN) 200 : full detail / (FR) 200 : détail complet	(EN) 200 : full detail / (FR) 200 : détail complet
GET /api/audits/{id}/ (other client's)	(EN) 404 : queryset excludes it / (FR) 404 : queryset l'exclut	(EN) 200 : admin sees all / (FR) 200 : admin voit tout
PATCH /api/audits/{id}/	(EN) 403 : PermissionDenied in update() / (FR) 403 : PermissionDenied dans update()	(EN) 200: can set status / notes. (FR) 200 peut définir statut / notes
DELETE /api/audits/{id}/	(EN) 403 : PermissionDenied in perform_destroy() / (FR) 403 : PermissionDenied dans perform_destroy()	(EN) 204 : sets status='archived' / (FR) 204 : passe status='archived'

6. Status Lifecycle & Notifications

6. Cycle de vie des statuts et notifications

The status field follows a defined lifecycle. Each status transition triggers an automatic email notification to the client via `create_and_send()`. Status is only updated by admins via `PATCH`. The `DELETE` endpoint is a pseudo-delete that sets `status='archived'` instead of removing the row.

Le champ status suit un cycle de vie défini. Chaque transition de statut déclenche une notification email automatique au client via `create_and_send()`. Le statut est mis à jour uniquement par les admins via `PATCH`. L'endpoint `DELETE` est un pseudo-delete qui passe `status='archived'` au lieu de supprimer la ligne.

Status	FR label	Who sets it	When / trigger EN + FR
pending	En attente	System (auto)	(EN) Set on creation by <code>AuditRequestCreateSerializer</code> (default). Notification <code>REQUEST_RECEIVED</code> sent to client. (FR) Défini à la création par le sérialiseur (défaut). Notification <code>REQUEST_RECEIVED</code> envoyée au client.
in_progress	En cours	Admin via <code>PATCH</code>	(EN) Admin starts working on the audit. Notification <code>STATUS_CHANGED</code> sent to client. (FR) L'admin commence l'audit. Notification <code>STATUS_CHANGED</code> envoyée au client.
completed	Termine	Admin via <code>PATCH</code>	(EN) Audit delivered. Admin may also set <code>completed_at</code> timestamp. (FR) Audit livré. L'admin peut aussi renseigner le timestamp <code>completed_at</code> .
archived	Archive	Admin via <code>DELETE</code>	(EN) Soft-delete. Row remains in DB. <code>perform_destroy()</code> sets <code>status='archived'</code> + saves only these fields. (FR) Soft-delete. La ligne reste en DB. <code>perform_destroy()</code> passe <code>status='archived'</code> + sauvegarde uniquement ces champs.

Notification triggers / Déclencheurs de notifications

```
# Trigger 1: on request creation (perform_create)
create_and_send(
    user=audit.client,
    request=audit,
    type_=Notification.Type.REQUEST_RECEIVED,
    subject=f'Audit request received - {audit.reference}',
    message=f'Hello {audit.client.first_name}, we received your request...',
)

# Trigger 2: on status change (perform_update)
if old_status != audit.status: # compare BEFORE and AFTER save
    create_and_send(
        user=audit.client,
        request=audit,
        type_=Notification.Type.STATUS_CHANGED,
        subject=f'Status updated - {audit.reference}',
        message=f'Status changed from "{old_status}" to "{audit.status}".',
    )

# Note: create_and_send() creates a Notification DB record AND sends email
```

7. Tests : 20 tests across 7 classes

7. Tests : 20 tests en 7 classes

The test suite covers both models and all endpoints. Key patterns: `admin_auth_client` fixture for admin-scoped tests, `make_client_user` factory for multi-user isolation tests, and UUID validation on the detail URL.

La suite de tests couvre les deux modeles et tous les endpoints. Patterns clés : fixture `admin_auth_client` pour les tests admin, fabrique `make_client_user` pour les tests d'isolation multi-utilisateurs, et validation UUID sur l'URL détail.

Class	Test	HTT P	Assertion EN / FR
TestAuditPackModel	test_four_packs_seeded	—	(EN) 4 packs with codes audit/security/protection/premium. (FR) 4 packs avec codes audit/security/protection/premium.
TestAuditPackModel	test_ordered_by_price	—	(EN) Prices list is sorted ascending. (FR) Liste des prix trie croissante.
TestAuditRequestModel	test_reference_auto_generated	—	(EN) reference starts 'DOSSIER-' and ends '-0001'. (FR) reference commence 'DOSSIER-' et finit '-0001'.
TestAuditRequestModel	test_default_status_is_pending	—	(EN) status == 'pending' after create. (FR) status == 'pending' apres create.
TestAuditRequestModel	test_two_creates_have_distinct_references	—	(EN) r1 != r2, and r2 ends '-0002'. (FR) r1 != r2, et r2 finit '-0002'.
TestPackListView	test_list_public_returns_200_and_4_packs	200	(EN) Unauthenticated GET returns 4 results in paginated response. (FR) GET non authentifie retourne 4 résultats pages.
TestAuditRequestCreate	test_client_can_create	201	(EN) status=='pending', reference starts 'DOSSIER-'. (FR) status=='pending', reference commence 'DOSSIER-'.
TestAuditRequestCreate	test_admin_cannot_create	403	(EN) Admin POST → 403 PermissionDenied. (FR) Admin POST → 403 PermissionDenied.
TestAuditRequestCreate	test_anonymous_returns_401	401	(EN) No auth → 401 Unauthorized. (FR) Sans auth → 401 Unauthorized.
TestAuditRequestList	test_client_sees_only_own_requests	200	(EN) 2 requests created (client + other) → client sees 1. (FR) 2

			demandes créées (client + autre) → client voit 1.
TestAuditRequestList	test_admin_sees_all_requests	200	(EN) 2 requests by different clients → admin sees 2. (FR) 2 demandes par clients différents → admin voit 2.
TestAuditRequestDetail	test_owner_can_read	200	(EN) Owner gets back their reference in response. (FR) Le propriétaire retrouve sa référence dans la réponse.
TestAuditRequestDetail	test_other_client_gets_404	404	(EN) Client B cannot see client A's request (queryset filtering). (FR) Client B ne peut pas voir la demande du client A (filtrage queryset).
TestAuditRequestDetail	test_admin_can_read_any	200	(EN) Admin reads any client's request detail. (FR) L'admin lit le détail de n'importe quel client.
TestAuditRequestUpdate	test_admin_can_patch_status	200	(EN) PATCH {status:'in_progress'}, DB value confirmed via refresh_from_db(). (FR) PATCH {status:'in_progress'}, valeur DB confirmée via refresh_from_db().
TestAuditRequestUpdate	test_client_cannot_patch	403	(EN) Client PATCH → 403 PermissionDenied from update(). (FR) Client PATCH → 403 PermissionDenied depuis update().
TestAuditRequestDelete	test_admin_archives_via_delete	204	(EN) DELETE → 204 AND req.status == 'archived' after refresh. (FR) DELETE → 204 ET req.status == 'archived' après refresh.
TestAuditRequestDelete	test_client_cannot_delete	403	(EN) Client DELETE → 403 PermissionDenied from perform_destroy(). (FR) Client DELETE → 403 PermissionDenied depuis perform_destroy().

8. Security & Design Summary

8. Recapitulatif securite et conception

Feature	Detail EN / FR
UUID primary key	(EN) UUIDField(editable=False) : prevents IDOR on request detail endpoint (no sequential IDs to enumerate). (FR) UUIDField(editable=False) prévient les IDOR sur l'endpoint détail (pas d'IDs séquentiels a énumérer).
Queryset scoping	(EN) Clients see only their own requests via. filter(client=request.user). Cross-client access returns 404 no information leakage. (FR) Les clients voient uniquement leurs demandes via. filter(client=request.user). Accès inter-clients → 404 pas de fuite d'information.
PROTECT FK	(EN) on_delete=PROTECT on client + pack prevents accidental data loss. Cannot delete a user with active audit requests. (FR) on_delete=PROTECT sur client + pack prévient la perte de données accidentelle. Impossible de supprimer un utilisateur avec des demandes actives.
Atomic reference generation	(EN) select_for_update() + transaction.atomic() guarantees unique references under concurrent load. (FR) select_for_update() + transaction.atomic() garantit des references uniques sous charge concurrente.
Soft delete (ARCHIVED)	(EN) DELETE sets status='archived' preserves audit trail, supports legal/compliance requirements. (FR) DELETE passe status='archived' préserve la piste d'audit, répond aux exigences légales/conformite.
internal_notes isolation	(EN) internal_notes only in AuditRequestAdminUpdateSerializer and AuditRequestSerializer (admin read). Not in create serializer. (FR) internal_notes uniquement dans AuditRequestAdminUpdateSerializer et AuditRequestSerializer (lecture admin). Pas dans le serialiseur de creation.
select_related (N+1 prevention)	(EN) get_queryset() uses select_related('pack','client','assigned_to') one SQL JOIN instead of N+3 queries per request. (FR) get_queryset() utilise select_related('pack','client','assigned_to') un seul JOIN SQL au lieu de N+3 requetes par demande.
Notification decoupling	(EN) create_and_send() in apps.notifications.services, audit logic and email logic are separate concerns. (FR) create_and_send() dans apps.notifications.services logique audit et logique email sont des responsabilites separees.